RWTH Aachen
Faculty of Computer Science
Chair of Combinatorial Optimization

Bachelor Thesis in Computer Science

Min-Max and Regret Spanning Trees

Complexity and Approximation under Discrete and Interval Uncertainty

Submitted by
Archit Dhama
Matriculation Number: 396580
archit.dhama@rwth-aachen.de

First Examiner
Prof. Dr. Christina Büsing
Chair of Combinatorial Optimization
RWTH Aachen

Second Examiner
Prof. Dr. Arie M. C. A. Koster
Chair of Discrete Optimization
RWTH Aachen

Aachen, May 13, 2026

Abstract

The minimum spanning tree (MST) connects all network nodes at minimum cost, but edge costs are often uncertain due to market fluctuations, estimation errors, or operational changes. This thesis studies robust spanning trees primarily under two uncertainty models (discrete scenarios and interval ranges) and two objectives (Min-Max, Min-Max Regret), classifying computational complexity and approximability across this design space; a third model, budgeted uncertainty, receives a focused complementary treatment.

We provide a self-contained treatment from first principles. After establishing MST foundations with five complete proofs (fundamental cycle and cut lemmas, three optimality criteria via exchange arguments), we systematically analyse the resulting problem variants. For intervals, we prove extremal lemmas showing worst cases occur at boundaries: Min-Max becomes polynomial while Regret remains NP-hard with a 2-approximation. For discrete scenarios, we prove weak NP-hardness for $K = 2$ via partition reduction and survey results for larger K including pseudo-polynomial algorithms and approximation schemes. The budgeted case is shown to inherit polynomial-time solvability for Min-Max through a citation-led reduction to nominal MST instances.

The thesis delivers nine complete proofs in total: five MST foundations, two representative hardness proofs (partition reductions for Min-Max and Regret with $K = 2$), and two approximation results (the 2-approximation algorithm with its supporting lemmas). A comprehensive classification table synthesises the complexity landscape. A fixed four-vertex micro-graph illustrates concepts throughout. This work balances mathematical rigour with pedagogical clarity, serving as a focused entry point into robust combinatorial optimisation.

Contents

Chapter 1

Introduction

1.1 Motivation

Consider the challenge of designing a fibre-optic telecommunications network connecting major cities across a region. The network must link all locations while minimising total cable installation cost. However, actual deployment costs depend on numerous factors: terrain characteristics (installing cables through mountainous regions costs substantially more than across plains), regulatory approvals that vary by jurisdiction, and material prices that fluctuate with global supply chains. Such cost uncertainty is pervasive in infrastructure planning; as Kouvelis and Yu [1] observe, forecasted parameter values rarely match realised costs in practice, motivating robust decision-making approaches. Consequently, a network design that appears optimal under today's cost estimates may become substantially more expensive if realised costs differ significantly. This uncertainty raises a fundamental question: how should one design infrastructure networks that remain near-optimal despite cost variations?

When edge costs are known precisely, the *minimum spanning tree* (MST) provides the classical solution. The MST connects all nodes at minimum total cost and can be computed efficiently using well-established algorithms such as Kruskal's greedy edge selection or Prim's incremental tree growing, both running in polynomial time. However, in practice, costs are rarely known with certainty at design time. A tree that is optimal for one cost estimate may perform poorly when costs deviate, potentially incurring substantially higher expenses or missing better alternatives. Thus, while the MST problem is well understood in the deterministic setting, real-world applications demand solutions that are *robust* to cost uncertainty.

Under uncertainty, two natural design objectives emerge. The first is the *Min-Max* objective, which seeks to minimise the worst-case absolute cost across all possible cost realisations. This conservative approach hedges against expensive scenarios by selecting a spanning tree whose cost remains controlled even in the most unfavourable circumstances. Mathematically, given a set of possible cost vectors, the goal is to find a tree that minimises the maximum cost it could incur.

The second objective is *Min-Max Regret*, which measures performance not in absolute terms but relative to what could have been achieved with perfect hindsight. Regret quantifies the difference between a chosen solution's cost and the optimal cost for a given realisation. The Min-Max Regret objective minimises the worst-

case performance gap, hedging against the possibility of missing the truly optimal solution. Unlike Min-Max, which guards against high absolute costs, Regret focuses on relative performance: ensuring that no matter which cost scenario materialises, the chosen tree is competitive with the scenario-optimal solution.

To model uncertainty, we consider two standard representations that arise naturally in applications. The first is *discrete scenario uncertainty*, where a planner enumerates a finite set of plausible cost vectors, each representing a different potential future state. For instance, in the telecommunications example, scenarios might correspond to different combinations of regulatory outcomes and material price levels. The second is *interval uncertainty*, where each edge cost is known to lie within a specified range, bounded by lower and upper estimates. This representation is particularly natural when costs are subject to continuous variation within known limits, such as fluctuations in commodity prices or exchange rates.

Together, these two objectives (Min-Max and Min-Max Regret) and the uncertainty models studied in this thesis (discrete scenarios, interval bounds, and budgeted uncertainty) define a design space of robust spanning tree variants. The central question addressed in this thesis is: how does computational complexity and approximability vary across this space? Which combinations admit polynomial-time algorithms, and which are computationally intractable? For the hard cases, what approximation guarantees can be achieved? These questions form the core of robust spanning tree optimisation and motivate the systematic classification developed in the following chapters.

1.2 Research Questions and Scope

This thesis addresses three core questions that arise naturally from the robust spanning tree framework outlined above.

First, what is the computational complexity of finding optimal robust spanning trees under the uncertainty models studied here? For each combination of objective (Min-Max or Regret) and uncertainty model (discrete, interval, or budgeted), does there exist a polynomial-time algorithm, or is the problem computationally intractable? When hardness arises, does it stem from the number of scenarios being unbounded, or is even the case of two scenarios already difficult? Answering these questions requires establishing precise complexity classifications: polynomial-time solvability, weak NP-hardness (admitting pseudo-polynomial algorithms), or strong NP-hardness (remaining hard even with unary-encoded inputs).

Second, where do worst-case costs occur within interval uncertainty? When each edge cost lies in a specified range, must one consider all points within the Cartesian product of intervals, or can worst cases be characterised more simply? This question is critical because if worst cases always occur at interval boundaries—such as the upper or lower bounds—then the infinite continuous uncertainty set can be reduced to a finite discrete problem. Extremal lemmas that identify such worst-case locations are therefore essential tools for both algorithmic design and theoretical analysis.

Third, for problems that are computationally hard, what approximation guarantees can be achieved? Can we design polynomial-time algorithms that provably compute solutions within a constant factor of optimal, or does the problem admit a fully polynomial-time approximation scheme (FPTAS) that achieves arbitrarily

good approximations in time polynomial in both problem size and accuracy? Conversely, are there hardness-of-approximation results that limit what can be achieved? These questions determine the boundary between tractable and intractable robust optimisation.

The scope of this thesis is deliberately focused. We restrict attention to spanning tree problems in undirected, connected graphs with static, single-stage decisions: a tree is selected once, before any uncertainty is resolved, and remains fixed. The uncertainty models studied are discrete scenarios (a finite collection of cost vectors), interval bounds (per-edge lower and upper limits forming a Cartesian product), and budgeted uncertainty (intervals with an additional cap on the total deviation, introduced briefly in Section 3.4). The objectives analysed are Min-Max (minimising worst-case absolute cost) and Min-Max Regret (minimising worst-case performance gap against scenario-optimal solutions).

This thesis does not cover polyhedral uncertainty sets, ellipsoidal uncertainty, or distributional robustness (where costs follow known probability distributions). It also excludes two-stage and recoverable optimisation models, where decisions can be partially adjusted after observing realisations. These extensions represent active research directions and are briefly discussed in the outlook (Section 5.4), but they lie outside the core scope of this work. The goal is not encyclopaedic coverage but rather a self-contained, rigorous treatment of the selected models, proving foundational results from first principles and synthesising established complexity and approximability findings into a unified classification.

1.3 Contributions

This thesis delivers five principal contributions to the literature on robust spanning tree optimisation.

First, we provide self-contained minimum spanning tree foundations (Chapter 2). We prove five fundamental results from first principles: the fundamental cycle and cut lemmas via exchange arguments, and three MST optimality criteria (cycle property, cut property, and their equivalence). Chapter 2 also includes a complexity primer defining polynomial time, NP-hardness (weak and strong), and approximation concepts (constant-factor algorithms, FPTAS, PTAS) for classifying the robust variants.

Second, we establish extremal characterisations for interval uncertainty (Chapters 3 and 4). For the Min-Max objective, Lemma 3 proves that worst-case costs occur when chosen edges are assigned their upper bounds, immediately yielding a polynomial-time algorithm. For the Regret objective, Lemma 4 proves that worst-case regret occurs at boundary vertices of the interval box, though determining which boundary is harder. Both lemmas are proved in full, and their implications for algorithm design are developed carefully.

Third, we provide two representative complexity proofs (Chapters 3 and 4). Theorem 4 establishes weak NP-hardness of Min-Max spanning trees with two discrete scenarios via a reduction from the Partition problem, constructing a grid graph where scenario costs encode target subset sums. The proof includes the reduction construction, correctness argument, and encoding analysis. Theorem 7 extends this to Min-Max Regret by reusing the construction with adjusted analysis, demonstrat-

ing that regret complexity mirrors absolute cost complexity for discrete scenarios. Together with cited results for larger scenario counts, these establish a complete complexity hierarchy.

Fourth, we prove a 2-approximation algorithm for interval regret spanning trees (Chapter 4). Theorem 11 demonstrates that solving the MST problem at midpoint costs yields a solution whose worst-case regret is at most twice optimal. The proof is developed rigorously through two supporting lemmas (Lemmas 5 and 6) bounding the midpoint solution's regret from below and above. This result is noteworthy as the best known constant-factor approximation for any robust combinatorial optimisation problem under interval uncertainty, though the tightness of the factor 2 remains an open question in the literature.

Fifth, we synthesise results into a comprehensive classification table (Chapter 5). **??** organises problem variants by objective and uncertainty model (discrete, interval, budgeted) with their complexity classes and approximation guarantees. The table reveals structural patterns: intervals and budgeted uncertainty exhibit extremal behaviour enabling tractability for Min-Max, scenario count K acts as a complexity parameter (constant versus unbounded) for discrete uncertainty, and the two objectives display parallel hardness for discrete scenarios but diverge for intervals.

Throughout the thesis, a fixed four-vertex micro-graph with interval costs, derived discrete scenarios, and derived budgeted parameters (nominal costs and maximum deviations) provides worked examples illustrating optimal solutions under each objective and uncertainty model. This pedagogical device grounds abstract theoretical results in concrete calculations, demonstrating that robust optima genuinely differ from nominal solutions.

1.4 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 establishes mathematical foundations, proving minimum spanning tree optimality criteria from first principles and defining the robust optimisation framework (uncertainty models, objectives, complexity terminology). A fixed micro-graph is introduced for illustrative examples throughout.

Chapter 3 analyses the Min-Max objective, characterising worst-case costs for interval uncertainty and establishing computational complexity for discrete scenarios across different values of the scenario count parameter.

Chapter 4 examines the Min-Max Regret objective, which measures relative performance rather than absolute cost. Interval regret is shown to be harder than interval Min-Max despite sharing extremal properties, while discrete regret mirrors the complexity hierarchy of discrete Min-Max.

Chapter 5 consolidates the results from Chapters 3 and 4 into a comprehensive classification table, displays the running micro-graph under each robust objective in an example gallery, acknowledges the deliberate scope limitations of this work, and points to related robust optimisation models (polyhedral uncertainty, two-stage formulations, recoverable robustness) along with open questions from the literature.

Chapter A provides an alphabetised notation table for quick reference.

Chapter 2

Foundations

This chapter develops the foundations needed for analysing robust spanning tree problems. After fixing graph-theoretic notation and introducing the running micrograph (Section 2.1), we prove the MST optimality criteria from first principles (Section 2.2) and remark on the classical greedy algorithms they justify (Section 2.3). Section 2.4 then formalises the three uncertainty models, discrete scenarios, interval uncertainty, and budgeted uncertainty, together with the min-max and min-max regret objectives. Section 2.5 reviews the complexity and approximation terminology used in Chapters 3 and 4.

2.1 Graphs, Trees, and Notation

We work exclusively with undirected graphs throughout this thesis. A *graph* is a pair $G = (V, E)$, where V is a finite non-empty set of *vertices* (or *nodes*) and $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is a set of *edges*. Each edge connects exactly two distinct vertices; we write $\{u, v\} \in E$ or equivalently $uv \in E$ to denote an edge between u and v. A graph is called *simple* if it contains no multiple edges (at most one edge between any pair of vertices) and no loops (edges connecting a vertex to itself). All graphs considered in this thesis are simple. We denote $n = |V|$ and $m = |E|$ for the number of vertices and edges, respectively. Two vertices $u, v \in V$ are *adjacent* (or *neighbours*) if $uv \in E$, and the *degree* of a vertex v, written $\deg(v)$, is the number of edges incident to v.

A *path* in G is a sequence of distinct vertices $v_0, v_1, \ldots, v_k$ such that $\{v_{i-1}, v_i\} \in E$ for all $i \in \{1, \ldots, k\}$. The *length* of this path is k (the number of edges). A *cycle* is a sequence of distinct vertices $v_0, v_1, \ldots, v_k$ with $k \geq 2$ such that $\{v_{i-1}, v_i\} \in E$ for all $i \in \{1, \ldots, k\}$ and $\{v_k, v_0\} \in E$.

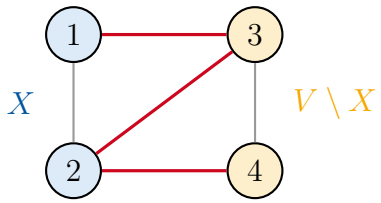
A graph G is *connected* if for every pair of vertices $u, v \in V$, there exists a path from u to v. A spanning tree can only exist if G is connected, since the tree must reach every vertex. **Throughout this thesis, we assume all graphs are connected.**

For a non-empty proper subset $X \subseteq V$ (that is, $\emptyset \neq X \subsetneq V$), the *cut* induced by X is the set of edges crossing the partition $(X, V \setminus X)$:

$$\delta(X) = \{\{u, v\} \in E \mid u \in X,\, v \in V \setminus X\}.$$

Intuitively, $\delta(X)$ comprises all edges with exactly one endpoint in X. See Figure 2.1

for a visual representation.



$$\delta(X) = \{\{1,3\},\{2,3\},\{2,4\}\}$$

Figure 2.1: Illustration of a cut $\delta(X)$ induced by vertex set $X = \{1,2\}$. The cut $\delta(X)$ consists of the three red edges connecting the blue-shaded vertices in X to the orange-shaded vertices in $V \setminus X$.

Each edge $e \in E$ has an associated *cost* $c_e \in \mathbb{R}$. We represent the cost structure either as a *cost function* $c\colon E \to \mathbb{R}$ or equivalently as a *cost vector* $c \in \mathbb{R}^{|E|}$ indexed by the edges. Costs may be positive, negative, or zero in the deterministic setting; the robust models in Section 2.4 will additionally impose non-negative costs. For any subset $F \subseteq E$, we write $c(F) = \sum_{e \in F} c_e$ for the total cost of the edge set F.

Spanning Trees. A *spanning tree* of G is a connected acyclic subgraph $T \subseteq G$ that includes all vertices of G, that is, $V(T) = V(G)$. We denote by $\mathcal{T}$ the set of all spanning trees of G. Since spanning trees are both connected and acyclic, they satisfy precisely $|E(T)| = n - 1$ edges, where $E(T) \subseteq E$ denotes the edge set of T. For a spanning tree $T \in \mathcal{T}$, we write $c(T) = c(E(T)) = \sum_{e \in E(T)} c_e$ for the cost of T.

The *minimum spanning tree problem* (MST) seeks a spanning tree of minimum cost:

$$\mathrm{MST}(c) = \min_{T \in \mathcal{T}} c(T). \tag{2.1}$$

Any tree $T^* \in \mathcal{T}$ satisfying $c(T^*) = \mathrm{MST}(c)$ is called an *MST* under cost vector c. The MST is unique if all edge costs are distinct; when edge costs have ties (equal values), multiple spanning trees may achieve the minimum cost $\mathrm{MST}(c)$.

A spanning tree T possesses three fundamental structural properties that we exploit repeatedly:

(P1) For any two vertices $u, v \in V$, there exists a *unique* path in T connecting u and v.

(P2) Removing any edge $e \in E(T)$ disconnects T into exactly two connected components.

(P3) Adding any edge $f \in E \setminus E(T)$ to T creates exactly one cycle.

Property P1 follows from the definition of trees as connected acyclic graphs: connectivity ensures path existence, while acyclicity guarantees uniqueness (a second path would form a cycle). Properties P2 and P3 are immediate consequences: removing an edge from a minimally connected subgraph (a tree on n vertices has exactly $n-1$ edges) must disconnect it, while adding an edge to a maximally acyclic subgraph must create a cycle [2, Theorem 2.1].

Fundamental Structures. We now introduce two graph-theoretic constructs, fundamental cycles and fundamental cuts, that play a central role in characterising MST optimality. Let $T \in \mathcal{T}$ be a spanning tree and let $f \in E \setminus E(T)$ be an edge not in T with endpoints u and v. By property P3, adding f to T creates a unique cycle. Since T is a tree, by property P1, there exists a unique path P in T from u to v. We call the cycle $C_f = E(P) \cup \{f\}$ (where $E(P)$ denotes the edges of path P) the *fundamental cycle* of f with respect to T.

Similarly, let $e \in E(T)$ be an edge in the tree T. By property P2, removing e from T partitions V into exactly two connected components. Let $X_e \subseteq V$ denote one of these components (the choice is arbitrary; either component induces the same cut). The *fundamental cut* of e with respect to T is the cut $\delta(X_e)$ induced by X_e. Crucially, e is the *only* tree edge in $\delta(X_e)$; all other edges in the cut belong to $E \setminus E(T)$. This follows because X_e is a connected component of $T \setminus \{e\}$: any other tree edge crossing the partition would connect X_e to $V \setminus X_e$, contradicting the component structure. These fundamental structures underpin the MST optimality criteria developed in Section 2.2: fundamental cycles correspond to potential edge exchanges for non-tree edges, while fundamental cuts correspond to exchanges for tree edges.

Running Example: The Micro-graph. To anchor the concepts of this thesis, we fix a four-vertex graph that recurs throughout as a running example. Let $G = (V, E)$ with vertex set $V = \{1, 2, 3, 4\}$ and five edges:

$$e_1 = \{1, 2\}, \quad e_2 = \{2, 3\}, \quad e_3 = \{2, 4\},$$
$$e_4 = \{3, 4\}, \quad e_5 = \{1, 3\}.$$

The structure is depicted in Figure 2.2. Vertex 2 serves as a central hub with edges to vertices 1, 3, and 4, while edges $e_5 = \{1, 3\}$ and $e_4 = \{3, 4\}$ provide alternative connections.

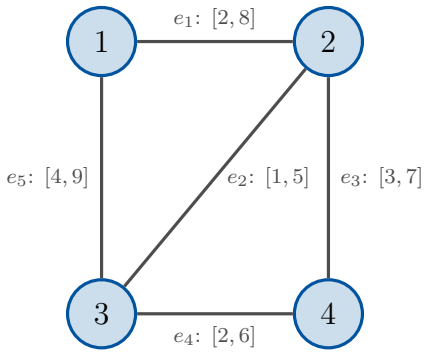
In anticipation of the robust optimisation framework of Section 2.4, we associate each edge e_i with an *interval cost* $[\ell_{e_i}, u_{e_i}]$ representing uncertainty in edge costs:

$$e_1 \colon [2, 8], \quad e_2 \colon [1, 5], \quad e_3 \colon [3, 7],$$
$$e_4 \colon [2, 6], \quad e_5 \colon [4, 9].$$

From these intervals, we derive three *discrete scenarios* that instantiate specific cost realisations: $c^{(1)} = (2, 1, 3, 2, 4)$ (all lower bounds), $c^{(2)} = (8, 5, 7, 6, 9)$ (all upper bounds), and $c^{(3)} = (5, 1, 7, 2, 4)$ (mixed costs). Graph G admits eight spanning trees in total. We highlight three representative trees that exhibit different structural properties: $T_1 = \{e_1, e_2, e_3\}$ (star centred at vertex 2), $T_2 = \{e_1, e_2, e_4\}$ (path 1–2–3–4), and $T_3 = \{e_2, e_3, e_5\}$ (path 1–3–2–4). These trees and scenarios provide a consistent testbed for demonstrating how optimal solutions vary across objectives and uncertainty models in subsequent chapters.

2.2 MST Optimality Criteria

A remarkable feature of minimum spanning trees is that optimality can be verified locally: rather than comparing a candidate tree against all exponentially many



Scenarios: $c^{(1)} = (2, 1, 3, 2, 4)$, $c^{(2)} = (8, 5, 7, 6, 9)$, $c^{(3)} = (5, 1, 7, 2, 4)$
Representative trees: $T_1 = \{e_1, e_2, e_3\}$, $T_2 = \{e_1, e_2, e_4\}$, $T_3 = \{e_2, e_3, e_5\}$

Figure 2.2: The micro-graph G with interval costs $[\ell_e, u_e]$ on each edge, used throughout this thesis.

spanning trees, we can check simple conditions on individual edges. This section develops these local characterisations through five results. We begin with two fundamental exchange lemmas (Lemmas 1 and 2) that establish necessary conditions for optimality, then prove three equivalent optimality criteria (Theorems 1 to 3) that provide both necessary and sufficient conditions, thereby fully characterising MST optimality. These characterisations underpin the correctness of classical MST algorithms such as Kruskal's and Prim's greedy methods.

Our first lemma relates non-tree edges to the cycles they create.

Lemma 1 (Fundamental Cycle Lemma)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a minimum spanning tree of G under c. For any edge $f \in E \setminus E(T)$, the edge f has maximum cost among all edges in its fundamental cycle C_f. That is, $c_f \geq c_e$ for every edge $e \in C_f$.*

Proof. Suppose for contradiction that f does not have maximum cost in C_f. Then there exists an edge $e \in C_f$ with $c_e > c_f$. Since $f \notin E(T)$ and C_f is created by adding f to T, the edge e must be a tree edge.

We construct an alternative spanning tree by exchanging e for f. Removing e from T disconnects T into two components; since e lies on the path connecting the endpoints of f in T, the edge f bridges these components. Define T' with edge set $E(T') = E(T) \setminus \{e\} \cup \{f\}$.

To verify that T' is a spanning tree, we check three properties. First, $|E(T')| = |E(T)| - 1 + 1 = n - 1$, as required for a tree on n vertices. Second, T' is connected: removing e partitions V into two components, but f crosses this partition (as f completes the cycle C_f containing e), so adding f reconnects the graph. Third, T' is acyclic: adding f to T created exactly one cycle (namely C_f), and removing $e \in C_f \setminus \{f\}$ eliminates it.

The cost of T' is

$$c(T') = c(T) - c_e + c_f < c(T),$$

where the inequality holds because $c_e > c_f$. This contradicts the optimality of T. Therefore, f must have maximum cost in C_f. □

The dual characterisation holds for tree edges via cuts.

Lemma 2 (Fundamental Cut Lemma)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a minimum spanning tree of G under c. For any edge $e \in E(T)$, the edge e has minimum cost among all edges in its fundamental cut $\delta(X_e)$. That is, $c_e \leq c_f$ for every edge $f \in \delta(X_e)$.*

Proof. Suppose for contradiction that e does not have minimum cost in $\delta(X_e)$. Then there exists an edge $f \in \delta(X_e)$ with $c_f < c_e$. By construction of the fundamental cut (Section 2.1), edge e is the unique tree edge in $\delta(X_e)$, so f must be a non-tree edge.

We construct an alternative spanning tree by exchanging e for f. Removing e from T splits V into components X_e and $V \setminus X_e$, which f bridges (since $f \in \delta(X_e)$). Define T' with edge set $E(T') = E(T) \setminus \{e\} \cup \{f\}$.

To verify that T' is a spanning tree, we check three properties. First, $|E(T')| = |E(T)| - 1 + 1 = n - 1$, as required for a tree on n vertices. Second, T' is connected: f has one endpoint in each component (by definition of $\delta(X_e)$), so adding f restores connectivity. Third, T' is acyclic: $T \setminus \{e\}$ is a forest of two trees, and adding a single edge between two trees produces a tree without cycles.

The cost of T' is

$$c(T') = c(T) - c_e + c_f < c(T),$$

where the inequality holds because $c_f < c_e$. This contradicts the optimality of T. Therefore, e must have minimum cost in $\delta(X_e)$. □

These fundamental exchange properties establish necessary conditions for a tree to be an MST. The optimality criteria developed next strengthen these to necessary and sufficient characterisations.

Remark 1 (Cost Ties). The fundamental lemmas use non-strict inequalities ($c_f \geq c_e$ in Lemma 1; $c_e \leq c_f$ in Lemma 2), since multiple edges may share the same cost. When all edge costs are distinct, the MST is unique and the inequalities can be tightened to strict; when ties exist, multiple spanning trees may achieve the minimum cost, and the lemmas apply to each of them.

Theorem 1 (Cycle Criterion)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T is a minimum spanning tree if and only if for every non-tree edge $f \in E \setminus E(T)$, the edge f has maximum cost among all edges in its fundamental cycle C_f.*

Proof. We prove both directions.

($\Rightarrow$): If T is a minimum spanning tree, then every non-tree edge has maximum cost in its fundamental cycle by Lemma 1.

($\Leftarrow$): Suppose every non-tree edge has maximum cost in its fundamental cycle. We show T is a minimum spanning tree by proving $c(T) \leq c(T')$ for any spanning tree T'.

If $T = T'$, we are done. Otherwise, there exists an edge $e \in E(T) \setminus E(T')$. Write $e = \{s, t\}$ and let $X_e \subseteq V$ denote the component containing s after removing e from T, so that $t \in V \setminus X_e$. Since T' is connected, there exists a path P in T' from s to t. Because s and t lie on opposite sides of the partition $(X_e, V \setminus X_e)$, the path P must cross it at least once.

Pick any edge f on P that crosses the partition. Then f has one endpoint in X_e and the other in $V \setminus X_e$, hence $f \in \delta(X_e)$. Moreover, $f \in E(T')$ (since f lies on the path P) and $f \notin E(T)$ (since e is the unique tree edge in $\delta(X_e)$ by construction of the fundamental cut; see Section 2.1).

We construct a new spanning tree $T'' = (V, E(T') \setminus \{f\} \cup \{e\})$ by removing f and adding e. To verify that T'' is a spanning tree, note that f lies on the path P from s to t in T', so removing f disconnects s from t, while adding $e = \{s, t\}$ reconnects them. Thus T'' is connected. Furthermore, the fundamental cycle of e in T' is precisely $P \cup \{e\}$, and since $f \in P$, removing f eliminates this unique cycle, ensuring T'' is acyclic. With $|E(T'')| = n - 1$ edges, T'' is a spanning tree.

We now verify that $c(T'') \leq c(T')$. Since $f \notin E(T)$, the edge f creates a fundamental cycle C_f in T. The path in T connecting the endpoints of f must cross the partition $(X_e, V \setminus X_e)$, and because e is the unique tree edge in $\delta(X_e)$, this path contains e. Therefore $e \in C_f$. By assumption, f has maximum cost in C_f, so $c_f \geq c_e$. Hence

$$c(T'') = c(T') - c_f + c_e \leq c(T').$$

Moreover, $|E(T) \cap E(T'')| = |E(T) \cap E(T')| + 1$, since we added $e \in E(T)$ and removed $f \notin E(T)$.

Repeating this exchange, we obtain a sequence of spanning trees $T' = T_0, T_1, T_2, \ldots$ where each T_{i+1} is derived from T_i by exchanging an edge of $E(T) \setminus E(T_i)$ with an edge of $E(T_i) \setminus E(T)$, satisfying $c(T_{i+1}) \leq c(T_i)$ and $|E(T) \cap E(T_{i+1})| = |E(T) \cap E(T_i)| + 1$. Since spanning trees have exactly $n - 1$ edges, the process terminates after at most $n - 1$ iterations with some $T_k = T$. The cost satisfies

$$c(T) = c(T_k) \leq c(T_{k-1}) \leq \cdots \leq c(T_1) \leq c(T_0) = c(T').$$

Therefore $c(T) \leq c(T')$ for every spanning tree T', which means T is a minimum spanning tree. □

The dual characterisation via cuts is analogous.

Theorem 2 (Cut Criterion)**.** *Let $G = (V, E)$ be a connected graph with cost function $c \colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T is a minimum spanning tree if and only if for every tree edge $e \in E(T)$, the edge e has minimum cost among all edges in its fundamental cut $\delta(X_e)$.*

Proof. We prove both directions.

($\Rightarrow$)**:** If T is a minimum spanning tree, then every tree edge has minimum cost in its fundamental cut by Lemma 2.

($\Leftarrow$)**:** Suppose every tree edge has minimum cost in its fundamental cut. We show $c(T) \leq c(T')$ for any spanning tree T'.

The argument mirrors the proof of Theorem 1. If $T = T'$, we are done. Otherwise, pick any edge $e \in E(T) \setminus E(T')$ with $e = \{s, t\}$, and let X_e denote the component containing s in $T \setminus \{e\}$.

Since T' is connected, there exists a path P in T' from s to t. Because $s \in X_e$ and $t \in V \setminus X_e$ lie on opposite sides of the partition, the path P crosses $(X_e, V \setminus X_e)$ at least once. Let f be any edge on P crossing the partition. Then $f \in \delta(X_e)$, $f \in E(T')$, and $f \notin E(T)$ (since e is the unique tree edge in $\delta(X_e)$ and $f \neq e$). By the cut criterion, $c_e \leq c_f$.

By the same spanning tree verification as in Theorem 1, the exchange $T'' = (V, E(T') \setminus \{f\} \cup \{e\})$ yields a spanning tree with

$$c(T'') = c(T') - c_f + c_e \leq c(T')$$

and $|E(T) \cap E(T'')| = |E(T) \cap E(T')| + 1$.

Repeating this exchange, we obtain a sequence $T' = T_0, T_1, \ldots, T_k = T$ where each step satisfies $c(T_{i+1}) \leq c(T_i)$ and increases $|E(T) \cap E(T_i)|$ by one. The process terminates when $T_k = T$, yielding

$$c(T) = c(T_k) \leq c(T_{k-1}) \leq \cdots \leq c(T_0) = c(T').$$

Therefore $c(T) \leq c(T')$ for every spanning tree T', so T is a minimum spanning tree. □

The two criteria are equivalent.

Theorem 3 (Equivalence of Criteria)**.** *Let $G = (V, E)$ be a connected graph with cost function $c\colon E \to \mathbb{R}$, and let T be a spanning tree of G. Then T satisfies the cycle criterion if and only if T satisfies the cut criterion.*

Proof. By Theorem 1, T satisfies the cycle criterion if and only if T is a minimum spanning tree. By Theorem 2, T satisfies the cut criterion if and only if T is a minimum spanning tree. Therefore, T satisfies the cycle criterion if and only if T satisfies the cut criterion. □

Together, these three theorems fully characterise minimum spanning tree optimality through local edge-level conditions, enabling verification without comparing against all exponentially many spanning trees. The characterisations extend naturally to the robust spanning tree problems in Chapters 3 and 4, where similar exchange arguments establish optimality under uncertainty. The correctness of classical greedy algorithms such as Kruskal's and Prim's follows from these criteria, as discussed in Section 2.3.

2.3 Kruskal and Prim: Algorithmic Remarks

The optimality criteria developed in Section 2.2 directly yield the correctness of the two most classical MST algorithms: Kruskal's algorithm (1956) and Prim's algorithm (1957). Both are *greedy* methods that build a spanning tree incrementally, maintaining optimality conditions throughout [3, §2.3]. We describe each algorithm briefly, emphasising its connection to the criteria rather than implementation details.

Kruskal's Algorithm. Kruskal's algorithm constructs an MST by processing edges in non-decreasing order of cost. Starting with an empty edge set, the algorithm considers each edge in turn and adds it to the current forest if and only if it does not create a cycle. The process terminates when the forest becomes a spanning tree (after adding exactly $n-1$ edges).

The correctness follows immediately from the cycle criterion (Theorem 1). When the algorithm terminates with spanning tree T, consider any non-tree edge $f \in E \setminus E(T)$. Since f was rejected, adding f would have created a cycle with edges already in T. All edges in this cycle were added before f was considered, hence each has cost at most c_f (by the sorting order). Therefore f has maximum cost in its fundamental cycle, satisfying the cycle criterion.

With an efficient union-find data structure to track connected components, Kruskal's algorithm runs in $O(m \log n)$ time, dominated by the initial edge sorting [2, Theorem 6.5].

Prim's Algorithm. Prim's algorithm grows a single tree from an arbitrary starting vertex. At each step, it adds the minimum-cost edge crossing the cut between the current tree vertices and the remaining vertices. The process continues until all vertices are included.

The correctness follows by induction on the steps of the algorithm. Let S_t denote the set of tree vertices before step t and e_t the edge added at step t. We claim that the partial tree built so far can be extended to some MST of G at every step; since the algorithm terminates with a spanning tree, this implies the final tree is itself an MST.

The base case is trivial: the empty tree on the chosen root extends to any MST. For the inductive step, assume the partial tree T_{t-1} is contained in some MST T^*, and consider the edge e_t added next. If $e_t \in E(T^*)$, the induction continues with the same T^*.

Otherwise $e_t \notin E(T^*)$. The unique path in T^* between the endpoints of e_t starts in S_{t-1} (one endpoint of e_t lies there) and ends outside S_{t-1} (the other endpoint is the newly added vertex), so it must cross the cut $\delta(S_{t-1})$ at some edge $f \in E(T^*)$. By Prim's choice, e_t is the minimum-cost edge in $\delta(S_{t-1})$, hence $c_{e_t} \leq c_f$. The exchange $T^{**} = (T^* \setminus \{f\}) \cup \{e_t\}$ is again a spanning tree: adding e_t to T^* creates exactly its fundamental cycle, and f lies on that cycle, so removing f restores acyclicity. Its cost satisfies $c(T^{**}) = c(T^*) - c_f + c_{e_t} \leq c(T^*)$, so T^{**} is also an MST and contains $T_{t-1} \cup \{e_t\}$.

By induction, the spanning tree built by Prim's algorithm is an MST.

With a priority queue maintaining the minimum-cost edge to each non-tree vertex, Prim's algorithm achieves $O(m+n^2)$ time with a simple array implementation, or $O(m + n \log n)$ with Fibonacci heaps [2, Theorem 6.7].

Relevance to Robust Problems. Both algorithms solve the deterministic MST problem optimally in polynomial time. However, under uncertainty, the situation changes fundamentally. When edge costs are uncertain, neither algorithm directly applies: the greedy choices depend on which cost realisation occurs, yet the spanning tree must be chosen before costs are revealed. The robust spanning tree problems

studied in Chapters 3 and 4 address precisely this challenge, and several variants become NP-hard despite the tractability of the deterministic case.

2.4 Uncertainty Models and Robust Optimisation

In many practical applications, edge costs are not known precisely at decision time. Construction expenses may fluctuate, travel times depend on traffic conditions, and communication link costs vary with demand. Robust optimisation addresses this challenge by seeking solutions that perform well across all possible cost realisations, rather than optimising for a single nominal scenario [1]. This section formalises three uncertainty models and the corresponding robust objectives that we analyse in Chapters 3 and 4.

Discrete Scenarios. In the *discrete scenario model*, uncertainty is represented by a finite set of possible cost vectors.

Definition 1 (Discrete Uncertainty Set)**.** *A discrete uncertainty set consists of K explicitly given cost vectors:*

$$\mathcal{U} = \{c^{(1)}, c^{(2)}, \ldots, c^{(K)}\},$$

where each scenario $c^{(k)}$, for $k \in \{1, \ldots, K\}$, is a cost vector in $\mathbb{R}^{|E|}$ assigning cost $c_e^{(k)}$ to every edge $e \in E$.

The parameter K denotes the number of scenarios. When K is a fixed constant (such as $K = 2$), we speak of a *constant number of scenarios*; when K is part of the input and may grow with the problem size, we speak of *unbounded* K. This distinction is crucial for computational complexity: several robust spanning tree problems are polynomial-time solvable for constant K but become NP-hard when K is unbounded [4].

Interval Uncertainty. In the *interval model*, each edge cost may take any value within a specified range.

Definition 2 (Interval Uncertainty Set)**.** *An interval uncertainty set is defined by lower and upper bounds on each edge:*

$$\mathcal{U} = \prod_{e \in E} [\ell_e, u_e] = \left\{ c \in \mathbb{R}^{|E|} : \ell_e \leq c_e \leq u_e \text{ for all } e \in E \right\},$$

where $0 \leq \ell_e \leq u_e$ for each edge $e \in E$.

This non-negativity reflects edge costs typically representing distances, construction expenses, or transmission delays, and is standard in the robust spanning tree literature [1, 5].

Unlike the discrete model with finitely many scenarios, interval uncertainty encompasses a continuum of possible cost vectors. Despite this, many robust problems under interval uncertainty admit efficient solutions by exploiting the structure of the uncertainty set: worst-case costs are often attained at the boundary of the intervals. We develop these arguments in Chapters 3 and 4.

Budgeted Uncertainty. In the *budgeted model*, each edge has a baseline cost together with a bound on how far it may rise, and a single parameter caps how many edges may deviate from baseline at once.

Definition 3 (Budgeted Uncertainty Set)**.** *Let* $\bar{c} \in \mathbb{R}_{\geq 0}^{|E|}$ *be a vector of* nominal costs *(the baseline value of each edge),* $\hat{c} \in \mathbb{R}_{\geq 0}^{|E|}$ *a vector of* maximum deviations *(the largest upward perturbation each edge can suffer), and* $\Gamma \in [0, |E|]$ *a* deviation budget*. The* budgeted uncertainty set *is*

$$\mathcal{U}^{\Gamma} = \left\{ c \in \mathbb{R}_{\geq 0}^{|E|} : c_e = \bar{c}_e + \hat{c}_e \, \delta_e, \;\; \delta_e \in [0,1] \text{ for } e \in E, \;\; \sum_{e \in E} \delta_e \leq \Gamma \right\}.$$

Each edge cost c_e is determined by a deviation factor $\delta_e \in [0,1]$: at $\delta_e = 0$ the edge takes its nominal cost $\bar{c}_e$, at $\delta_e = 1$ it takes the worst-case value $\bar{c}_e + \hat{c}_e$, and intermediate values interpolate linearly. The budget constraint $\sum_e \delta_e \leq \Gamma$ limits the cumulative deviation across the graph. Setting $\Gamma = 0$ recovers the deterministic problem on $\bar{c}$, while $\Gamma = |E|$ allows every edge to deviate independently and recovers an interval model with $[\ell_e, u_e] = [\bar{c}_e, \bar{c}_e + \hat{c}_e]$ [5]. The budget Γ thus offers a tunable middle ground: the modeller controls how conservative the analysis is through a single parameter, rather than committing in advance to a fixed scenario list or to the full interval ranges. As we shall see in Section 3.4, this added structure allows the min-max spanning tree problem to remain polynomial-time solvable under budgeted uncertainty.

Robust Objectives. Given an uncertainty set $\mathcal{U}$, we model robust optimisation as a sequential game: a decision-maker first selects a spanning tree T, then an adversary selects a worst-case scenario $c \in \mathcal{U}$. This structure motivates two robust objectives.

The *min-max objective* minimises the maximum cost over all scenarios:

$$\min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} c(T). \tag{2.2}$$

A spanning tree achieving this minimum is called a *min-max spanning tree*. This objective bounds the worst possible outcome regardless of which scenario materialises.

The *min-max regret objective* instead minimises the maximum *regret*, which measures how far a solution falls short of the scenario-specific optimum:

$$\text{Regret}(T, c) = c(T) - \text{MST}(c). \tag{2.3}$$

The min-max regret problem seeks a tree minimising the worst-case regret:

$$\min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} \text{Regret}(T, c) = \min_{T \in \mathcal{T}} \max_{c \in \mathcal{U}} \Big[c(T) - \text{MST}(c) \Big]. \tag{2.4}$$

A spanning tree achieving this minimum is called a *min-max regret spanning tree*. This objective minimises *opportunity cost*: the loss from not knowing the scenario in advance.

Computing regret requires knowing $\mathrm{MST}(c)$, which depends on the scenario c. For discrete scenarios, we can precompute the MST for each scenario separately. For interval uncertainty, the adversary's choice of c affects both $c(T)$ and $\mathrm{MST}(c)$ simultaneously, and the worst-case scenario is generally not simply all upper bounds; identifying it requires the extremal arguments developed in Chapters 3 and 4.

Worked Example: Discrete Scenarios on the Micro-graph. We illustrate the robust objectives using the discrete scenario model on the micro-graph from Section 2.1. (A worked example for interval uncertainty requires the extremal analysis developed in Chapters 3 and 4.)

The micro-graph has vertices $V = \{1, 2, 3, 4\}$, edges $\{e_1, \ldots, e_5\}$, and interval costs as specified in Figure 2.2. We derive three discrete scenarios: $c^{(1)} = (2, 1, 3, 2, 4)$ (all lower bounds), $c^{(2)} = (8, 5, 7, 6, 9)$ (all upper bounds), and $c^{(3)} = (5, 1, 7, 2, 4)$ (mixed). We evaluate three spanning trees: $T_1 = \{e_1, e_2, e_3\}$, $T_2 = \{e_1, e_2, e_4\}$, and $T_3 = \{e_2, e_3, e_5\}$.

Table 2.1 records the cost and regret for each tree–scenario pair. The bottom row shows the MST cost for each scenario; note that the MST varies: T_2 is optimal under $c^{(1)}$ and $c^{(2)}$, but under $c^{(3)}$ a different tree $T^* = \{e_2, e_4, e_5\}$ achieves the minimum cost 7.

Table 2.1: Costs and regrets for representative spanning trees under three discrete scenarios.

	Cost $c^{(k)}(T)$			Regret			Max
Tree	k=1	k=2	k=3	k=1	k=2	k=3	Regret
$T_1 = \{e_1, e_2, e_3\}$	6	20	13	1	1	6	6
$T_2 = \{e_1, e_2, e_4\}$	5	19	8	0	0	1	1
$T_3 = \{e_2, e_3, e_5\}$	8	21	12	3	2	5	5
$\mathrm{MST}(c^{(k)})$	5	19	7				

To find the min-max regret tree, we proceed in two steps: compute each tree's maximum regret across scenarios (rightmost column), then select the tree with the smallest maximum. Tree T_2 has max regret 1, compared to 6 for T_1 and 5 for T_3, making T_2 the min-max regret spanning tree.

For the min-max objective, we compare worst-case costs: T_2 achieves 19 (under $c^{(2)}$), versus 20 for T_1 and 21 for T_3. Thus T_2 is also the min-max spanning tree. In this example, both objectives select the same tree, though this coincidence does not hold in general.

2.5 Algorithmic Preliminaries

This section introduces the complexity and approximation terminology needed to interpret the results in Chapters 3 and 4. We focus on concepts directly relevant to robust spanning tree problems; for comprehensive treatments, see [5, Chapter 2] or [2, Chapter 15].

Polynomial Time and the Class P. An algorithm runs in *polynomial time* if, given an input of size n, it terminates in at most $O(n^k)$ steps for some fixed constant k. The class P consists of all problems solvable by a polynomial-time algorithm. Problems in P are considered tractable: their running time grows manageably with the input size. The minimum spanning tree problem lies in P, since Kruskal's algorithm solves it in $O(m \log n)$ time (Section 2.3). Introducing uncertainty, however, can push problems outside P, as we shall see in Chapters 3 and 4.

Hard Problems: NP and NP-hardness. The class NP (nondeterministic polynomial time) contains problems whose candidate solutions can be *verified* in polynomial time, even if finding such a solution may itself be hard. To make this concrete, consider the decision version of MST: given a graph $G = (V, E)$, costs c, and a budget B, does there exist a spanning tree of cost at most B? A candidate solution is a set $T \subseteq E$ of $n-1$ edges. Verifying it requires two checks: that T is a spanning tree (one breadth-first search on the $n-1$ edges of T runs in $O(n)$ time and confirms connectedness), and that the sum of its edge costs does not exceed B (another $O(n)$ additions). Since both checks are polynomial, the decision MST lies in NP.

A problem is NP-*hard* if every problem in NP can be *reduced* to it: transformed in polynomial time so that solving the target problem also solves the original. Informally, NP-hard problems are at least as hard as the hardest problems in NP. No polynomial-time algorithm is known for any NP-hard problem, and the widely believed conjecture $\mathsf{P} \neq \mathsf{NP}$ implies that none exists.

Weak and Strong NP-hardness. The knapsack problem is NP-hard, yet a textbook dynamic-programming algorithm solves it in $O(nW)$ time, where W is the knapsack capacity. At first sight this seems to contradict $\mathsf{P} \neq \mathsf{NP}$. The resolution lies in how input size is measured: a number W is encoded in binary using only about $\log_2 W$ bits, not W itself. Doubling W therefore adds just one bit to the input but doubles the runtime, so $O(nW)$ is polynomial in the *value* of W but exponential in the *bit-length* of W, which is what "polynomial-time" strictly demands.

An algorithm is *pseudo-polynomial* if its running time is polynomial in the input size and in the magnitudes of the numeric values appearing in the input. Such an algorithm is not polynomial in the strict sense, but it is efficient whenever those values are not too large. This motivates two refinements of NP-hardness:

- A problem is *weakly* NP-*hard* if it is NP-hard and admits a pseudo-polynomial algorithm. The knapsack problem is the canonical example.
- A problem is *strongly* NP-*hard* if it is NP-hard even when all numeric values are bounded by a polynomial in n. Such problems admit no pseudo-polynomial algorithm unless $\mathsf{P} = \mathsf{NP}$.

This distinction is significant for robust spanning trees: the number of scenarios K plays a role analogous to a numeric input value, and several variants will turn out to be weakly NP-hard (Chapters 3 and 4).

Approximation Algorithms. When an exact polynomial-time algorithm is unlikely to exist, we settle for efficient algorithms returning provably near-optimal solutions.

An *α-approximation algorithm* for a minimisation problem is a polynomial-time algorithm that, on every instance, returns a solution of cost at most α times the optimum, where $\alpha \geq 1$ is the *approximation ratio*. The ratio is fixed for the algorithm: a 2-approximation guarantees a solution at most twice as costly as the optimum, with no tighter promise. Christofides' algorithm for the metric travelling salesman problem, for instance, is a $\frac{3}{2}$-approximation [2, Theorem 21.5].

For some problems, the ratio can instead be tuned by the user. A *polynomial-time approximation scheme* (PTAS) takes any $\varepsilon > 0$ as input and returns a $(1+\varepsilon)$-approximation in time polynomial in n for each fixed ε. Setting $\alpha := 1 + \varepsilon$, this is precisely an α-approximation for any $\alpha > 1$ chosen by the user, with smaller ε yielding tighter guarantees. The catch is that the runtime may grow rapidly in $1/\varepsilon$ (for instance, $O(n^{1/\varepsilon})$), so a PTAS need not be practical for very small ε.

A *fully polynomial-time approximation scheme* (FPTAS) is a PTAS whose running time is additionally polynomial in $1/\varepsilon$. The trade-off then becomes graceful: halving ε at most polynomially increases the runtime. The knapsack problem admits an FPTAS [2, Theorem 17.9], illustrating that even some NP-hard problems can be approximated to within any desired precision in practical time. However, strongly NP-hard problems cannot have an FPTAS unless P = NP.

Relevance to This Thesis. The complexity of robust spanning tree problems hinges on two factors:

1. **Number of scenarios** K: For discrete uncertainty, many problems are in P when K is constant but become NP-hard when K is part of the input.
2. **Uncertainty model**: For interval uncertainty, the continuum of scenarios prevents enumeration. Some variants remain in P by exploiting MST structure (Section 2.2); others are NP-hard.

Chapters 3 and 4 classify each problem variant accordingly and present polynomial-time algorithms or approximations as appropriate.

Summary

Two technical pillars established in this chapter recur throughout the rest of the thesis. The first is the local characterisation of MST optimality through fundamental cycles and cuts (Theorems 1 and 2): the exchange arguments built on these criteria will reappear repeatedly in the robust setting. The second is the family of uncertainty models (discrete scenarios, interval uncertainty, and budgeted uncertainty) combined with robust objectives (min-max cost and min-max regret), which yields the problem variants analysed in Chapters 3 and 4. Several of these variants will turn out to be NP-hard, in sharp contrast to the polynomial-time tractability of the deterministic MST.

Chapter 3

Min-Max Spanning Tree

3.1 Problem Formulation

3.2 Interval Worst-Case Characterisation

Lemma 3 (Interval Extremal Cost)**.**

Proof. □

3.3 Complexity and Approximation

3.3.1 Discrete Scenarios: Constant K

Theorem 4 (Partition Reduction)**.**

Proof. □

Theorem 5 (Pseudo-Polynomial Algorithm)**.**

3.3.2 Discrete Scenarios: Unbounded K

Theorem 6 (Strong NP-Hardness)**.**

3.4 Budgeted Uncertainty

3.5 Discussion

Summary

Chapter 4

Min-Max Regret Spanning Tree

4.1 Regret Formulation

4.2 Interval Regret Extremal Behaviour

Lemma 4 (Interval Extremal Regret)**.**

Proof. □

4.3 Complexity for Discrete Scenarios

4.3.1 Constant K

Theorem 7 (K=2 Weak NP-Hardness)**.**

Proof. □

Theorem 8 (Pseudo-Polynomial for Constant K)**.**

4.3.2 Unbounded K

Theorem 9 (Strong NP-Hardness)**.**

4.4 Interval Regret Approximation

Theorem 10 (Interval NP-Hardness)**.**

Theorem 11 (2-Approximation via Midpoint)**.**

Lemma 5 (Lower Bound on Regret)**.**

Proof. □

Lemma 6 (Upper Bound Relating Solutions)**.**

Proof. □

Proof of Theorem 11. □

4.5 Discussion

Summary

Chapter 5

Conclusion

5.1 Synthesis of Results

Complexity Landscape.

Key Patterns.

5.2 Example Gallery

Micro-Graph Solutions Across Objectives.

Extremal Behaviour Visualisation.

5.3 Limitations

Scope of Uncertainty Models.

Scope of Solution Concepts.

Proof Selection.

5.4 Outlook

Extensions in Uncertainty Modelling.

Extensions in Solution Concepts.

Open Problems.

Appendix A

Notation

This appendix provides a reference for the mathematical notation used throughout the thesis. Symbols are grouped by category with the section of first use indicated.

Symbol	Meaning	First Use
Graph Notation		
$G = (V, E)$	Undirected graph with vertex set V, edge set E	§2.1
$n = \|V\|$, $m = \|E\|$	Number of vertices and edges	§2.1
$\delta(X)$	Cut induced by vertex set $X \subseteq V$	§2.1
Trees and Costs		
$\mathcal{T}$	Set of all spanning trees of G	§2.1
T	Spanning tree, $T \in \mathcal{T}$	§2.1
$c\colon E \to \mathbb{R}$	Edge cost function	§2.1
$c(T)$	Cost of tree T, i.e., $\sum_{e \in T} c_e$	§2.1
$\mathrm{MST}(c)$	Minimum spanning tree cost under c	§2.1
T^*	A minimum spanning tree, $c(T^*) = \mathrm{MST}(c)$	§2.1
C_f	Fundamental cycle of non-tree edge f	§2.1
$\delta(X_e)$	Fundamental cut of tree edge e	§2.1
Uncertainty Models		
$\mathcal{U}$	Uncertainty set (discrete, interval, or budgeted)	§2.4
K	Number of discrete scenarios	§2.4
$c^{(k)}$	Scenario k cost vector, $k \in \{1, \ldots, K\}$	§2.4
$[\ell_e, u_e]$	Interval cost bounds for edge e	§2.4
ℓ_e, u_e	Lower and upper bounds on edge cost (interval model)	§2.4
$\mathcal{U}^\Gamma$	Budgeted uncertainty set	§2.4
Γ	Deviation budget (budgeted model)	§2.4
$\bar{c}_e$	Nominal cost of edge e (budgeted model)	§2.4
$\hat{c}_e$	Maximum deviation of edge e (budgeted model)	§2.4

Continued on next page

Symbol	Meaning	First Use
Robust Objectives		
$\mathrm{Regret}(T, c)$	Regret of T under c: $c(T) - \mathrm{MST}(c)$	§2.4
Complexity		
P	Polynomial-time complexity class	§2.5
NP	Nondeterministic polynomial-time class	§2.5
NP-hard	At least as hard as NP-complete problems	§2.5
Weakly NP-hard	NP-hard with a pseudo-polynomial algorithm	§2.5
Strongly NP-hard	NP-hard even with poly-bounded numeric values	§2.5
Pseudo-polynomial	Algorithm polynomial in input size and numeric magnitudes	§2.5
α-approximation	Algorithm guaranteeing $\mathrm{ALG} \leq \alpha \cdot \mathrm{OPT}$	§2.5
ε	Tolerance parameter for approximation schemes	§2.5
PTAS	Polynomial-time approximation scheme	§2.5
FPTAS	Fully polynomial-time approximation scheme	§2.5

Bibliography

[1] Panos Kouvelis and Gang Yu. *Robust Discrete Optimization and Its Applications*. Vol. 14. Nonconvex Optimization and Its Applications. Dordrecht: Kluwer Academic Publishers, 1997. ISBN: 978-0-7923-4291-5. DOI: 10.1007/978-1-4757-2620-6.

[2] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. 6th ed. Vol. 21. Algorithms and Combinatorics. Berlin, Heidelberg: Springer, 2018. ISBN: 978-3-662-56039-6. DOI: 10.1007/978-3-662-56039-6.

[3] Christina Büsing. "Combinatorial Optimization – Optimierung B (Skript OptiB)". Lecture notes, Version: February 4, 2023. RWTH Aachen, 2023.

[4] Hassene Aissi, Cristina Bazgan, and Daniel Vanderpooten. "Min–max and min–max regret versions of combinatorial optimization problems: A survey". In: *European Journal of Operational Research* 197.2 (2009), pp. 427–438. DOI: 10.1016/j.ejor.2008.09.012.

[5] Marc Goerigk. *An Introduction to Robust Combinatorial Optimization: Concepts, Models and Algorithms for Decision Making under Uncertainty*. Cham: Springer, 2021. DOI: 10.1007/978-3-030-82392-5.

[6] Dimitris Bertsimas and Melvyn Sim. "Robust discrete optimization and network flows". In: *Mathematical Programming* 98.1–3 (2003), pp. 49–71. DOI: 10.1007/s10107-003-0396-4.

Affidavit / Eidesstattliche Versicherung

I hereby declare that I have written this thesis independently and that I have not used any sources or aids other than those indicated. All passages which are quoted or closely paraphrased from other works are marked as such. This thesis has not been submitted in the same or a substantially similar form to any other examination board.

Place, Date Signature